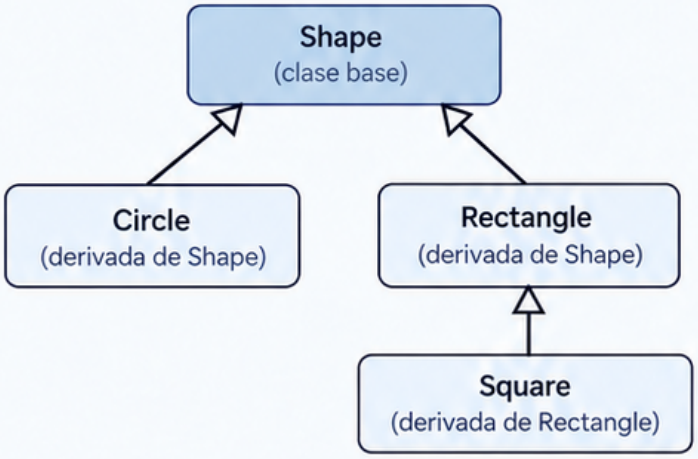
Polimorfismo con Shape

El mismo recorrido calcula áreas de objetos distintos

C++20 • Shape, Circle, Rectangle y Square.

La jerarquía del ejemplo

Square tiene una base directa: **Rectangle**. La cadena hasta **Shape** es herencia multinivel, no múltiple.



Un solo recorrido

```cpp
Circle c{5.0};
Rectangle r{4.0, 6.0};
Square s{3.0};

Shape* figuras[]{&c, &r, &s};

for (const Shape* figura : figuras) {
    std::cout << figura->area() << '\n';
}
```

Salida con dos decimales

```
78.54
24.00
9.00
```

Orden de salida: Circle, Rectangle, Square.
El programa completo fija la precisión con `std::fixed` y `std::setprecision(2)`.

Todas comparten la operación

Shape declara:

```cpp
virtual double area() const = 0;
virtual ~Shape() = default;
```

Circle redefine: pi × radio × radio.
Rectangle redefine: base × altura.
Square redefine: lado × lado.
Las tres implementaciones usan `const override`.

Cálculo de área en cada clase

`Circle::area()` : pi × radio × radio

`Rectangle::area()` : base × altura

`Square::area()` : lado × lado

Fragmentos didácticos: las clases completas están en `ejemplos/`.

Durante la construcción

Una llamada virtual desde un constructor o destructor se resuelve en la clase que está construyéndose o destruyéndose; no salta a una parte derivada que todavía no está activa.

Límites del modelo

Esta jerarquía es didáctica. Si Rectangle permite cambiar base y altura por separado, derivar Square puede romper ese contrato. Revisa los invariantes antes de reutilizar esta relación en otro diseño.

C++20: [class.virtual], [class.cdtor]. C++ Core Guidelines C.120.